



DL Latest updates: <https://dl.acm.org/doi/10.1145/3664476.3664513>

RESEARCH-ARTICLE

SECURA: Unified Reference Architecture for Advanced Security and Trust in Safety Critical Infrastructures

MICHAEL ECKEL, Fraunhofer Institute for Secure Information Technology SIT, Darmstadt, Hessen, Germany



I am a cyber security researcher for Fraunhofer SIT. Previously, I was a security engineer at Huawei Technologies, mobile software developer at Boostix/GAL, and a web and software developer for a number of other companies. I hold a master's degree in Computer Science. Currently, I am co-chairing the Trusted Computing Group's Network Equipment work group, working to secure vulnerable network equipment. My research interests include cyber security, Trusted Computing, virtualization technologies, and distributed systems.

SIGRID GÜRGENS, Fraunhofer Institute for Secure Information Technology SIT, Darmstadt, Hessen, Germany

Open Access Support provided by:

Fraunhofer Institute for Secure Information Technology SIT

Published: 30 July 2024

Citation in BibTeX format

ARES 2024: The 19th International Conference on Availability, Reliability and Security
July 30 - August 2, 2024
Vienna, Austria

SECURA: Unified Reference Architecture for Advanced Security and Trust in Safety Critical Infrastructures

Michael Eckel
Fraunhofer SIT | ATHENE Center
Darmstadt, Hesse, Germany
michael.eckel@sit.fraunhofer.de

Sigrid Gürgens
Fraunhofer SIT | ATHENE Center
Darmstadt, Hesse, Germany
sigrid.gurgens@sit.fraunhofer.de

ABSTRACT

In the evolving landscape of safety-critical infrastructures, ensuring the integrity and security of systems has become paramount. Building upon a previously established security architecture tailored for the railway sector, this work introduces significant enhancements that extend its applicability beyond the confines of any singular industry. Key advancements include the integration of a security heartbeat to augment safety monitoring, the implementation of a sophisticated secure update mechanism leveraging Trusted Platform Module (TPM) Enhanced Authorization (EA) policies, local TPM based attestation, a cyber-resiliency watchdog in a Trusted Execution Environment (TEE) that detects compromised system components and triggers remediation actions, automated vulnerability scanning leveraging Linux Integrity Measurement Architecture (IMA) logs to check against vulnerability databases, and a formal evaluation of system integrity reporting capabilities through remote attestation.

Moreover, aiming for a universally adaptable framework, this paper proposes a reference architecture to accommodate various operational contexts. We use compartments—such as virtual machines, software containers, and RTOS partitions—as a universal abstraction for system components, designed to be compatible with various real-time operating systems (RTOSes), including the safety-certified PikeOS, the ACRN hypervisor, and beyond.

CCS CONCEPTS

• **Security and privacy** → **Trusted computing; Virtualization and security; Distributed systems security**; • **Hardware** → **Safety critical systems**.

KEYWORDS

safety, security, critical infrastructures, remote attestation, local attestation, cyber-resiliency watchdog, secure update, Trusted Computing, Confidential Computing

ACM Reference Format:

Michael Eckel and Sigrid Gürgens. 2024. SECURA: Unified Reference Architecture for Advanced Security and Trust in Safety Critical Infrastructures. In *The 19th International Conference on Availability, Reliability and Security*

(ARES 2024), July 30–August 02, 2024, Vienna, Austria. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3664476.3664513>

1 INTRODUCTION

Information Technology (IT) and network connectivity are advancing into critical infrastructures and merge with safety-critical Operational Technology (OT), where hardware and software is used for detecting or causing changes in physical processes through direct monitoring and/or control of physical devices. The landscape of system architecture is evolving, moving away from the traditional proprietary, monolithic, and costly systems towards the adoption of standard hardware and software technologies, commercial off-the-shelf (COTS) products, and IP-based communication. This shift towards IT solutions not only fosters increased efficiency but also significantly reduces costs. However, the increased connectivity and the use of COTS components also enable new attacks, especially if systems are deployed in the field where an attacker may have physical access. It is therefore necessary to integrate suitable IT security measures into safety-critical OT systems.

In safety-critical OT areas such as railway systems, safety certifications are prescribed for the systems used. However, security solutions cannot usually be certified under safety conditions. On the one hand, the threats are not deterministic and are constantly evolving. On the other hand, many operations for cryptographic procedures are not deterministic, such as the generation of random numbers or cryptographic keys. Even if a threat analysis according to IEC 61508 has been carried out and security requirements have been identified, it is not possible to define security measures analogous to safety measures that comply with certain assumptions such as Mean Time Between Failures (MTBF). A central challenge is therefore the integration of security measures while maintaining safety certifications.

In this paper, we introduce SECURA, a reference architecture that incorporates a TPM along with associated security measures into a safety-critical OT systems. The target system implements basically four security (and safety) components: a RTOS, a hardware security module in the form of a TPM 2.0, and various security services, such as measured boot, remote attestation, secure update, and vulnerability scanning based on Linux IMA logs. Safety applications, e. g., those certified to Safety Integrity Level (SIL) 4, and security applications with a lower SIL can coexist on the same platform in separate compartments, ensuring the security applications do not compromise the safety applications' performance and integrity. Our architecture is based on the works in [17, 29, 32] but concentrates on the TPM aspects and the use of the TPM for ensuring system integrity. We discuss implementation aspects and evaluate how our architecture addresses the relevant security and



This work is licensed under a Creative Commons Attribution International 4.0 License.

ARES 2024, July 30–August 02, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1718-5/24/07
<https://doi.org/10.1145/3664476.3664513>

safety requirements. The key aspect of ensuring system integrity with TPM based remote attestation is also formally analyzed. Our main contributions are:

- (1) Definition of a universally applicable reference architecture for safety-critical systems, based on a previously established security architecture tailored for the railway sector [17, 32].
- (2) Augmenting safety monitoring with a security heartbeat for system integrity via TPM based remote attestation.
- (3) Integration of local integrity verification through TPM-based attestation within a Trusted Execution Environment (TEE), ensuring continuous local monitoring of software and hardware integrity.
- (4) Development of a cyber-resiliency watchdog that utilizes local attestation results to detect compromised system components and initiate appropriate remediation actions, such as recovery or selective compartment restart.
- (5) Formal evaluation of the remote attestation process.
- (6) Development of a secure update mechanism for compartments, using TPM Enhanced Authorization (EA) policies.
- (7) Automated vulnerability monitoring using Linux IMA logs to check against a comprehensive vulnerability database.

The paper is organized as follows. In Section 2, we discuss related work. We describe our attacker model in Section 3, and derive security and safety requirements in Section 4. We introduce reference architecture in Section 5. Furthermore, we briefly discuss implementation aspects in Section 6. Our evaluation is described in Section 7, followed by the formal evaluation for remote attestation in Section 8. We conclude our paper in Section 9.

2 RELATED WORK

In the realm of safety-critical systems, various international standards govern the development and certification processes across different sectors to ensure reliability, safety, and security. Key standards include IEC 61508 (functional safety), DO-178B/C (aviation), ISO 26262 (automotive), UNECE R 155 (automotive cybersecurity), EN 50126 (IEC 62278), EN 50128, EN 50129, and DIN VDE 0831-104 (railway), IEC 62443 (industrial communication networks), and IEEE 1547 (energy systems). Together, these standards play a crucial role in guiding the development, testing, and certification of systems and software across different domains, ensuring that they meet the highest levels of safety and reliability.

In the energy sector, the primary standards in the USA include IEEE 1547 for interconnecting distributed energy resources (DERs) and IEEE 2030.5 for the cybersecurity of DER communications. In Europe, the key standards are IEC 61850, which describes the native protocols for substation automation such as Generic Object Oriented Substation Event (GOOSE) and Sampled Values (SV), the general Manufacturing Message Specification (MMS) protocol, and a derivative of the Distributed Network Protocol 3 (DNP3). These are secured by the IEC 62351 standard, which because of time constraints, allows some exceptions for GOOSE. Another relevant protocol is IEC 60870-104, originally part of the DNP3.

In terms of safety, security, and resilience engineering, the United States has published the NIST SP 800-160 standard, which comprises two volumes. Volume 1 defines and describes systems security engineering, complementing ISO/IEC/IEEE 15288, which covers

systems and software engineering and system life cycle processes. Building on this, Volume 2 defines and standardizes cyber resilience.

The European Union Cyber Resilience Act (CRA) mandates cyber resilience, among other requirements, aiming to enhance the overall cybersecurity of network and information systems across the EU. This includes stringent measures for managing vulnerabilities, incident response, and ensuring the resilience of critical infrastructure.

DIN VDE 0831-104 [12] served as the foundational framework in [29] for conducting a threat and risk analysis aimed at identifying security requirements for safety-critical railway signaling networks. Additionally, research has introduced hardware-based security platforms for railway Control, Command, and Signaling (CCS) systems, facilitating the integration of uncertified security functions alongside certified safety functions on a single hardware device without affecting the latter's certification [7, 29].

In other related work, Schlehuber et al. discussed approaches for integrating security mechanisms in safety-critical systems without affecting the safety-relevant parts of the system [36]. A security analysis of the Rail Safe Transport Application (RaSTA) safety protocol for railway signaling, which also provides some basic security mechanisms, and how they could be improved is presented in [30]. Using multicore processors for a railway safety and security concept for low-power mixed-criticality systems has been proposed in [5]. A safety concept for a railway mixed-criticality embedded system based on multicore partitioning where different safety-levels can be executed on the same multicore platform while maintaining safety-certification has been proposed by Agirre et al. [1].

There are a couple of approaches that address weaknesses of cryptographic algorithms used in railway signal protocols and suggest methods of improvement (cf. [29, 33, 41]). In contrast, our approach provides the guarantee that any component supposed to apply a cryptographic algorithm can be trusted to behave correctly via TPM based attestation.

The use of TPMs to secure embedded systems is currently the subject of intensive research, e. g., for industrial control systems [4] and automotive systems [23]. The TPM-based secure update protocol proposed in [24] is used as basis for our update concept. The developed remote attestation concept is based on the principles of remote attestation defined in [9].

3 ATTACKER MODEL

We consider an attacker trying to get control over the target system. The attacker can attack either the communication channel between the target system and backend, or the target system itself. We assume the backend is secure.

For attacking the target system, we assume a powerful attacker who has full physical access to the target system. It is aligned with the cyber-physical Dolev-Yao attacker [35] that extends the classic Dolev-Yao attacker [13].

The attacker can modify or replace components, install malicious firmware, and extract any stored information, except for tamper protected information, e. g., in a TEE or Hardware Security Modules (HSMs), such as the TPM. Possible attack vectors include offline attacks against flash/storage and runtime attacks against Random-Access Memory (RAM), carried out with or without installation of

compromised firmware or exploits. Further, the attacker can eavesdrop, replay, inject, relay, modify, and/or intercept all transmitted messages, and may execute side channel attacks, read, modify or delete security-sensitive memory. However, the attacker cannot break any cryptography or circumvent the protection guarantees by HSMs, including TEEs.

4 SECURITY AND SAFETY REQUIREMENTS

Our main goal is to protect the target system against the cyber-physical Dolev-Yao attacker described in Section 3 while maintaining safety functionalities. Thus, certain security and safety requirements must be met. Building on the security requirements identified in [17, 29], we further refine these into more detailed requirements, categorizing them into safety and security dimensions.

4.1 Security Requirements

- SeR₁ Secure key generation:* Cryptographic keys required for securing the target system as well as the communication shall be generated within a protected and secure environment. This can be either in the backend system or in a special security module within the target system.
- SeR₂ Secure storage:* Critical data such as (private) keys shall also be stored in the protected and secure environment to prevent their leakage. Especially the target system should provide some specially protected area for secure key storage.
- SeR₃ Secure cryptographic operations:* To protect critical cryptographic keys from leakage during usage, they should also be used within a protected and secure environment. Especially the target system should provide some specially protected area for secure cryptographic operations.
- SeR₄ Key usage authorization:* To prevent arbitrary access to critical keys by an unauthorized service of the system or a tampered firmware, access to these keys shall be limited to authenticated and trusted systems.
- SeR₅ Secure communication channel:* Integrity, authenticity, and confidentiality of the communication between target system and backend shall be ensured.
- SeR₆ Remote integrity verification:* The target system should only run authorized and trusted software instances. Since this cannot be always guaranteed, the backend shall be able to remotely verify the system integrity of the target system, i. e., verify whether the software has not been tampered with and is in a trustworthy state.
- SeR₇ Local integrity verification:* The system shall support local attestation to continuously monitor and verify the integrity of software and hardware components, ensuring that any deviation from known-good states is detected promptly.
- SeR₈ Cyber-resiliency watchdog:* The system shall include a cyber-resiliency watchdog that can autonomously take action based on the results of local attestation, such as performing remediation, recovery, or selective restarts to maintain operational integrity and safety.
- SeR₉ Secure remote software update:* The backend shall be able to remotely install software updates on the system deployed in the field. Integrity, authenticity, and confidentiality of the software update shall be ensured. In addition, the system

deployed in the field shall be able to prevent an attacker from installing an older software version (downgrade/rollback attack).

4.2 Safety Requirements

- SaR₁ Separation:* The system shall support spatial and temporal separation of applications running on the same hardware platform to enable the integration of safety and security applications on the same hardware platform.
- SaR₂ Computational redundancy:* Hardware platforms that provide computational redundancy shall be supported, i. e., safety-critical operations are computed independently on at least two different compute units (CUs), e. g., CPUs.
- SaR₃ Communication redundancy:* The communication between the backend and the target system shall be able to provide redundancy, i. e., at least two physical communication paths can be used.
- SaR₄ Fulfillment of timing constraints:* The system shall be able to perform all safety-critical operations within the defined time limits, i. e., security operations shall not affect the correct operation of safety functions.
- SaR₅ Safety certification:* The system shall be able to receive a safety certification (despite added security functions).

5 SECURA ARCHITECTURE

Our architecture allows non-safety certified security measures to be integrated with safety certified OT systems. It features strong hardware identities, remote integrity verification, and hardened secure software updates. Figure 1 depicts our SECURA architecture.

The backend consists of the several components. The Safety Control and Monitoring is responsible for controlling and monitoring safety compartments. Further, it assesses the security state with the help of attestation results from Remote Attestation, and reacts to those, e. g., instructing the target system to enter failsafe mode. Moreover, the backend features a Secure Update to securely install compartment updates from remote.

The Remote Attestation component receives and appraises attestation evidence from the target system (cf. [8]). For the appraisal, it requires certificates (or public keys) from TPM keys as well as reference values of known good software versions and configurations. The Local Attestation component, running in a protected and isolated TEE on the target system, also requires these keys and certificates as well as the reference values; that is why they need to be synchronized (Sync component in the backend) between the two. Further, the Remote Attestation collects Boot Logs, Compartment Logs, and Linux IMA Logs that we use in the Vulnerability Monitoring to check entries against vulnerabilities to identify vulnerable software.

The SECURA architecture’s backend configuration could potentially integrate or connect with a Security Information and Event Management (SIEM) system or a Security Operations Center (SOC). Nonetheless, our proposed architecture does not explicitly incorporate these elements, affording adopters the flexibility to determine the inclusion of additional security components as per their requirements.

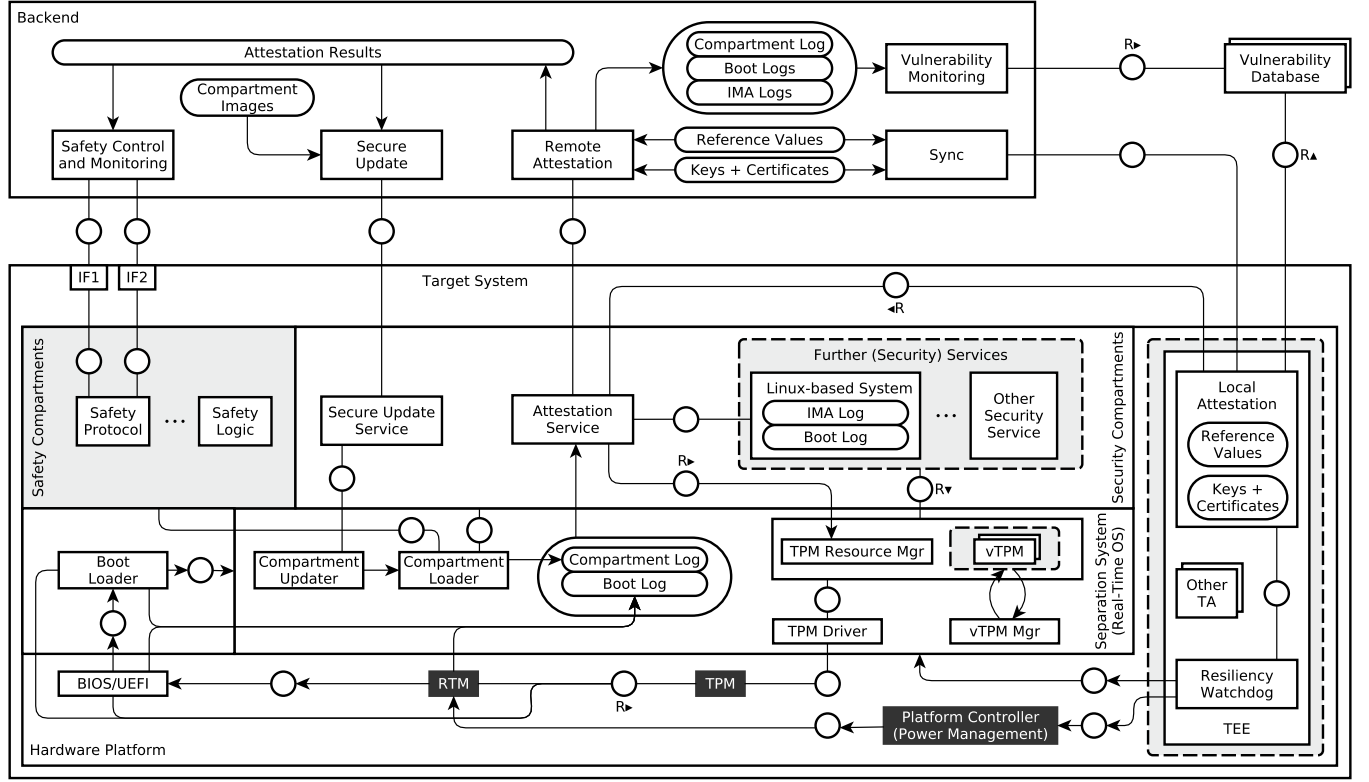


Figure 1: SECURA Reference Architecture for Advanced Security and Trust in Safety Critical Infrastructures

The target system is connected to the backend. The hardware platform contains a TPM, a cryptographic coprocessor in hardware that is hardened against physical attacks. It provides secure storage and shielded execution of cryptographic operations. A TPM provides a True Random Number Generator (TRNG), which is essential for secure generation of cryptographic keys. Keys can be securely stored in the TPM in a way that never leave the TPM (unencrypted). In addition, the TPM supports several security services such as measured boot. For measured boot, a Root of Trust for Measurement (RTM) is required which is an immutable code executed first after starting a system. TPM and RTM are Trusted Building Blockss (TBBs), i. e., the parts of the system that must be trusted implicitly.

In contrast to central processing units (CPUs), TPMs cannot be designed redundantly by their very nature [38]. For example, TPMs must generate and use random numbers. Therefore, two TPMs will always be in different states and generate different results.

However, the TPM as well as the accompanying TPM Software Stack (TSS) are not safety certified. As a consequence, we split the concept in such a way that only a small certifiable part of the TSS needs to be integrated into the RTOS separation kernel. All other parts run in functionally distinct compartments. This way, safety and security applications can coexist on the same platform, strongly isolated in their own compartments. As a result, security applications cannot interfere with safety applications [7, 29].

When the system is started, first the RTM is executed, then the BIOS/UEFI, and then Boot Loader that starts the RTOS. The RTOS is responsible for loading compartments, either safety or security compartments, and for updating compartments. At boot-time, a log of the started components, and at runtime a compartment log of started (and stopped) compartments is generated.

We realize security compartments for remote attestation and secure update. However, additional security compartments can be added as appropriate, such as intrusion detection or network anomaly detection. Figure 1 further depicts two separate communication channels fulfilling safety requirements for redundancy. We assume that Inter Process Communication (IPC) channels between compartments are secure. Shared memory regions between compartments must be explicitly defined. As a result, no software side channel attacks are possible.

Further security services may contain Linux-based systems, such as ELinOS for PikeOS or Linux virtual machines (VMs) for the ACRN hypervisor. To abstract from concrete implementations, we include a TPM Resource Manager (RM) based on the TPM Access Broker and Resource Manager (TABRM) from the Trusted Computing Group (TCG). It handles concurrent access from multiple compartments to the TPM, thus enabling additional compartments to use the TPM. Further, we introduce Virtual TPMs (vTPMs), that are managed by the vTPM Manager, and are connected to VMs.

They serve the same purpose as the physical TPM, but offer benefits about scalability. As parts of the RTOS, vTPMs are included in the compartment log and anchored in the physical TPM.

5.1 Real-Time Operating Systems

PikeOS stands out with its microkernel-based RTOS architecture designed for safety and security-critical sectors, featuring a Multiple Independent Levels of Security (MILS) approach for stringent isolation. ACRN offers a specialized, lightweight hypervisor aimed at IoT and embedded systems, prioritizing real-time operations and safety-critical virtualization. FreeRTOS provides a highly efficient, scalable RTOS for embedded devices, renowned for its simplicity and widespread use in low to medium complexity projects. Zephyr further complements this suite with its highly configurable and modular design, supporting multiple hardware architectures and is designed for connectivity-intensive IoT applications. Together, these technologies form a robust foundation for developing secure, reliable, and high-performance embedded systems and IoT solutions, making them valid choices for our SECURA architecture.

Containerized systems, designed for efficiency and scalability, generally lack the stringent safety and real-time capabilities required for safety-critical applications. Their architecture, prioritizing deployment speed over rigorous isolation, makes them unsuitable for contexts where failure poses significant risks.

5.2 Remote Attestation

Remote attestation is a process in which a Verifier checks the integrity of a target system, the Attester (cf. [8]). For that purpose, the Attester provides evidence about its current operational state. The Verifier verifies the evidence, by making sure that only authenticated software is executed on the Attester. As a result, the Verifier yields an assertion about the trustworthiness of the Attester. In our architecture, the Remote Attestation component of the backend represents the Verifier, and the Attestation Service of the target system represents the Attester.

5.2.1 Measured Boot. Evidence must originate from trusted system components to be meaningful. In our security architecture (cf. Fig. 1) we rely on Measured Boot to maintain a Boot Log of all started software. The log includes measurements of the software in the form of secure cryptographic hashes. We protect the integrity of that Boot Log with the TPM, representing the Root of Trust for Storage (RTS) in our system.

A TPM enables integrity protection of event logs. For that purpose, it implements Platform Configuration Registers (PCRs). PCRs are used to anchor log entries by providing a cryptographic folding hash function, called *TPM2_PCR_Extend*. This allows for continuously *extending* hashes of log entries into PCRs. PCRs can neither be reset nor be set to arbitrary values. They can only be *extended* and read. This way, tamper-resistant logging is possible. An attacker cannot wipe her traces undetected, e. g., by trying to replay a log without entries that would disclose her malicious activities.

The Measured Boot process starts with the RTM. The RTM must be implicitly trusted as stated above. After it has executed its main logic, it measures the next component in the boot sequence, the Basic Input/Output System (BIOS) or Unified Extensible Firmware Interface (UEFI). It records the measurement in the Boot Log, and

extends the hash of the Boot Log entry to a PCR in the TPM. Only then, control is passed to the UEFI. This principle of *first measure, then start* repeats along system components, i. e., RTM, UEFI, Boot Loader, and eventually the RTOS Kernel. After the kernel has started, the boot phase is considered done, and the runtime of the system begins.

The Compartment Loader is responsible for measuring and starting all safety and security compartments. It is part of the Separation Kernel, and as such already measured. Whenever a compartment is to be started, the Compartment Loader measures it, logs it into the Compartment Log, and anchors the log entry in the TPM. Exactly as before, with one significant difference: The RTOS Kernel stays in control and does not pass it to another component.

The number of possible compartments may be fixed, i. e., defined statically at compile time. Otherwise, safety critical timing constraints may not be guaranteed. A compartment can be shut down, though, and be replaced by another compartments, e. g., when a compartment is updated. The Compartment Loader then measures the updated compartment before it is started. That is why the Compartment Log advances during runtime, while the Boot Log does not.

Measured Boot records measurements of components and securely anchors them in the TPM, before they become active. This also holds for compromised components, which would be detected as such in a remote attestation process. Measured Boot in this respect forms the basis for remote attestation.

5.2.2 Evidence Reporting. Evidence must be reported in trustworthy manner from the Attester to the Verifier [6, 8]. This requires authenticity and integrity of evidence data. We use the TPM as Root of Trust for Reporting (RTR) to fulfill that requirement [38].

Further, the Verifier needs latest evidence from the Attester to evaluate its *current* integrity. Hence, replay attacks must be mitigated, in order not to fool a Verifier into using old evidence. To fulfill these requirements, the Verifier, i. e., the Remote Attestation component of the backend generates a random nonce. This nonce is used in an attestation request to the Attestation Service of the target system. The Attestation Service uses that nonce to perform a *TPM Quote* operation on the TPM [39]. A *TPM Quote* is a TPM issued signature over its PCRs, including the nonce, and current time information about the TPM internal clock. Data is signed using a TPM resident Attestation Key (AK) key pair. The public part AK_{pub} is known to the Backend in the form of a certificate and the private part AK_{priv} is securely stored in the TPM.

The Remote Attestation Service reads the Boot Log and the Compartment Log, both of which are anchored in PCRs. The logs, as well as the TPM Quote, are then transferred back to the backend. The nonce in that case provides both freshness/recentness of information and an effective countermeasure to replay attacks.

We secure the communication between the backend and the target system with a TPM-backed TLS channel, which provides confidentiality, authenticity, and integrity of the communication.

5.2.3 Evidence Verification. After the Verifier receives the attestation result from the Attester, it verifies whether the target system booted up correctly (boot-time) as well as whether it started only authentic compartments during its operation (runtime). Boot-time attestation encompasses all components that are executed in

strictly sequential order during system boot, i. e., up to the operating system (OS) kernel. From there, the runtime of a system begins, executing software applications in parallel, in unpredictable, non-deterministic order.

The Verifier maintains a reference values that represent known good system configurations. By comparing them to the current operational state of the Attester, it decides whether a system is considered to be trustworthy.

Having available the attestation result (TPM Quote, Boot Log, and Compartment Log) and reference values, the verification encompasses the following steps:

- (1) Verify the signature of the TPM Quote with the AK_{pub} .
- (2) Validate Boot Log and Compartment Log:
 - (a) Recompute the folding hash digest from the log.
 - (b) Compare the recomputed folding hash digest to the corresponding PCR value(s) from the TPM Quote, ensuring the log has not been tampered with.
 - (c) Compare the contents of the log with the reference values, i. e., compare measurement hashes from the log with expected hashes from the reference value.

As a result, the Verifier emits an attestation result, i. e., an assertion about the integrity and trustworthiness of the Attester. In our proof-of-concept (PoC) implementation, we leverage and adapt the open-source CHALLENGE-Response based Remote Attestation with TPM 2.0 (CHARRA) project [19] to meet our specific requirements and enhance its capabilities to align with our system’s needs.

For the integrity verification process, among others, Software Bills of Materials (SBOMs) can be utilized as reference values. SBOMs provide a comprehensive inventory of software components, their versions, and their dependencies, serving as a baseline for validating the integrity of the software running on remote devices. This approach aligns with the directives of Executive Order 14028, which emphasizes improving the nation’s cybersecurity through enhanced transparency and accountability in software supply chains.

Looking ahead, Hardware Bills of Materials (HBOMs) may also be used as reference values to ensure the integrity of hardware components, further enhancing the robustness of the attestation process. There is ongoing research on explicitly measuring hardware integrity, as demonstrated by the ASCOT project [40], with several research works published on this topic [14, 31, 37].

5.3 Security Heartbeat

Safety protocols like the RaSTA protocol [30], provide a heartbeat to periodically ensure availability of systems. Accordingly, we introduce a heartbeat to ensure the integrity of systems, using remote attestation. The backend’s remote attestation component periodically sends attestation requests to the target system. In case the integrity of a target system is detected not to be trustworthy, the target system may have been compromised by an attacker. Alternatively, unidirectional attestation protocols, such as Time-Based Uni-Directional Attestation (TUDA) [22], can be used where the target system regularly sends attestation messages to the backend, without requiring the backend to request these.

We introduce a change of behavior to the backend, by including integrity information from the Remote Attestation component in decisions for the Safety Control and Monitoring component. If either of the heartbeats fail, the Safety Control and Monitoring component sets the target system to failsafe mode, i. e., out of operation. With the double heartbeat, we ensure availability and integrity of target systems regularly, integrating safety and security monitoring.

5.4 Local Attestation and Resiliency Watchdog

In many safety-critical systems, maintaining operational status and defending against detected attacks is imperative, even when communication with the backend is lost. This is where local attestation comes into play. It continuously monitors the system’s software integrity (and potentially also hardware integrity, cf. Section 5.2.3) via local attestation. This local attestation is considered a privileged security function that is preferably implemented as a trusted application (TA) in a TEE due to the TEE’s security guarantees. It should be noted that a TEE may not provide real-time functionality. However, Arm Cortex-R processors, including the Cortex-R52 and Cortex-R82, do provide real-time capabilities in the TrustZone (Arm’s TEE solution) [2, 3]. These processors are specifically designed for real-time and safety-critical applications, such as automotive and industrial systems, making them suitable for environments where deterministic performance and reliability are crucial.

Local attestation needs to regularly synchronize the known-good reference values, as well as the keys and certificates, with the backend. It can also directly query vulnerability databases to identify compromised software versions.

We also propose a cyber-resiliency watchdog that takes the locally produced attestation results as input and determines necessary actions, such as remediation, recovery, or a simple reset of the entire platform to restore it to a stable operational state—referred to as “cyber safe position (CSP)” by the authors of [25]. Attack detection is not limited to what Trusted Computing-based remote attestation provides. Additional security services—either a TA in the TEE or an ordinary security compartment—such as (AI/ML based) host and network-based intrusion and anomaly detection, may be defined and considered by the watchdog for its decisions.

In some cases, it may not be possible or permissible to trigger a full system restart due to safety or regulatory requirements. However, it may be feasible to selectively restart or shut down compromised compartments, such as VMs, software containers, and partitions of a microkernel OS.

5.5 Secure Update

In our architecture, we consider updates to security compartments vital, to protect against new attacks or to apply bug fixes. Safety certification prohibits changes to certified safety compartments, so we are required to constrain updates to security compartments only. Compartment updates are deployed as monolithic binaries which replace a compartment as a whole—for ACRN this is a VM image.

Our goal for secure updates is to achieve hardware rooted confidentiality, integrity, and authenticity. For that purpose, we use a TPM protected asymmetric key pair to encrypt updates, the Secure

Update Key (SUK). The public part of the key pair (SUK_{pub}) is available to the Secure Update component of the backend. Another key pair is used to digitally sign the update, the Secure Update Signing Key (SUSK). The private part of the key pair ($SUSK_{priv}$) remains in the Secure Update component of the backend. The public part of the key pair ($SUSK_{pub}$) is available to the Secure Update Service component of the target system. Our secure update process is as follows:

- (1) The backend generates a random one-time symmetric key:

$$symKey \xleftarrow{R} \mathbb{G}$$

- (2) and encrypts the compartment update $ComptUpd$ with it:

$$cc = Enc(symKey, ComptUpd)$$

- (3) Then, the backend encrypts the $symKey$ with the SUK_{pub} , resulting in:

$$sc = Enc(SUK_{pub}, symKey)$$

- (4) The backend signs the encrypted symmetric key sc as well as a hash of the encrypted compartment update $h(cc)$ with the $SUSK_{priv}$ ($h()$ denotes a cryptographic hash function, e. g., SHA512, and $||$ denotes a concatenation):

$$sig = Sign(SUSK_{priv}, h(cc||sc))$$

- (5) The backend transfers cc , sc , and sig to the Secure Update Service of the target system.

- (6) The Secure Update Service verifies the signature:

$$v = Verify(SUSK_{pub}, sig, h(cc||sc)); v \in \{valid, invalid\}$$

- (7) If the verification is successful, the Secure Update Service decrypts the symmetric key sc with the SUK_{priv} in the TPM:

$$symKey = Dec(SUSK_{priv}, sc)$$

- (8) Then, the Secure Update Service decrypts the compartment update cc with the symmetric key $symKey$, which results in the decrypted $ComptUpd$:

$$ComptUpd = Dec(symKey, cc)$$

Eventually, the Secure Update Service sends the decrypted compartment update to the Compartment Updater, which shuts down the appropriate compartment, applies the update, and starts the updated compartment.

Before a secure update is transmitted, we ensure with remote attestation that the target system is in a trustworthy state. After the secure update has been applied, we use remote attestation again to ensure it has been applied properly. For more details on this aspect see our proof sketch in Section 8.5.

We further secure SUK usage by protecting it with TPM EA policies, which enable access to a key only if the specified policies are fulfilled. In the Backend, we create an asymmetric key pair, the Policy Authorize Key (PAK). The public part of that key is stored in the Secure Update compartment on the target system. We protect the SUK with a $TPM2_PolicyAuthorize$ policy, which allows the Backend to sign a new policy so that it can be used in an existing policy [39]. This allows the Backend to craft and sign arbitrary new policies which are enforced by the TPM, when access to the SUK is required.

We use the $TPM2_PolicyAuthorize$ mechanism to craft a tailored usage policy for each software update individually. We consider the Backend secure. An attacker cannot gain access to the PAK key, and hence cannot sign new policies. In particular, we protect key usage with the following policies.

Platform Integrity We restrict key usage to a particular platform operational state, by binding key usage to particular PCRs with $TPM2_PolicyPCR$. From the latest remote attestation we know the *good* PCR values, and can craft an appropriate policy (cf. [24]).

Expiration Time With the policy $TPM2_PolicyCounterTimer$, we make sure that a key expires after a defined time. For that purpose, we extract the TPM internal time and clock information from the latest remote attestation. We add the defined time on top of it, e. g., 5 minutes, and craft the $TPM2_PolicyCounterTimer$ policy with the condition: *less than* expiration time. This makes the key valid only if the current TPM time is less than the expiration time.

Rollback Protection We additionally secure compartment updates against rollback attacks, using the $TPM2_PolicyNV$ policy. We create a monotonic counter in the TPM NVRAM for every compartment. Each compartment update increments the counter. The Backend keeps track of all compartment updates and thus, knows the value of the appropriate monotonic counter. The Backend then crafts a $TPM2_PolicyNV$ policy which requires the monotonic counter in the TPM to be *lower* than the counter that the Backend knows (cf. [24]). This way, an attacker cannot replay an older software update.

Operation Restriction With the $TPM2_PolicyCpHash$ policy command, we restrict key usage to a specific TPM command, e. g., $TPM2_RSA_Decrypt$, as well as specific arguments, i. e., the symmetric key sc . This also ensures the integrity of the symmetric key because it includes a hash of the TPM command code along with the argument (sc), and all of this is signed with the PAK.

With these measures, we achieve that the usage of the key in the TPM can only be triggered by the Backend, for a limited time, and for the particular action. No other TPM operation can be performed, and after 5 minutes key usage renders impossible.

5.6 Vulnerability Scanning

The Linux IMA logging mechanism captures crucial data, including hashes and file paths of all executed OS software. This dataset underpins the backend Vulnerability Monitoring process, enabling the assessment of known software weaknesses and vulnerabilities by querying vulnerability databases.

However, for querying the Common Vulnerabilities and Exposures (CVE) database effectively, mere hash and file path information prove insufficient due to the database's requirement for detailed software version data. Consequently, without a comprehensive software inventory, effective CVE database consultation is unattainable. In our PoC implementation (cf. Section 6), we leverage VirusTotal's services, which permit database queries using just a software binary's hash digest. This approach facilitates the identification of the binary via its hash, yielding a compilation of findings from various sources that classify the software into categories such as malicious,

suspicious, undetected, and harmless. Further, VirusTotal provides additional information for binaries, including the software version, that can further be used to directly query the CVE database for known vulnerabilities.

6 IMPLEMENTATION

In our PoC explorations, we focus on developing individual components of the security services and demonstrating their integration within a specific architectural framework tailored to a particular application scenario. In contrast, the work presented in [17] details the incorporation of remote attestation alongside an early version of secure update mechanisms within the PikeOS environment, specifically in the context of the railway sector.

Our efforts differ as we concentrate on developing the missing elements related to secure updating (cf. Section 5.5) and vulnerability monitoring (cf. Section 5.6) across separate PoC constructs. The artifacts generated from our research are made accessible for scholarly examination at <https://github.com/eckelmeckel/paper-secura>. Furthermore, the encapsulation and fortification of VMs have been systematically detailed and implemented in [15], providing a solid foundation for securing virtual environments.

7 EVALUATION

Our evaluation demonstrates that the security requirements outlined in Section 4 are effectively addressed by the SECURA architecture. By using a TPM, we achieve secure key generation, secure storage, and secure cryptographic operations inside the TPM. This fulfills security requirements SeR_1 , SeR_2 , and SeR_3 . The backend is considered secure (cf. Section 4).

We use a TPM-backed TLS channel between the backend and the target system, providing confidentiality, integrity, and authenticity of the communication, thus, fulfilling security requirement SeR_5 .

With remote attestation as well as our safety & security heartbeat, we fulfill security requirement SeR_6 . The integrity verification via local attestation SeR_7 is achieved through continuous monitoring of software (and hardware) components. This ensures that any deviations from the known-good state are promptly detected, thereby maintaining system integrity.

The cyber-resiliency watchdog SeR_8 enhances the system’s ability to remain operational under attack conditions by autonomously deciding and executing necessary actions based on local attestation results. This includes actions such as remediation, recovery, or selective compartment restarts, thereby ensuring that the system can recover to a stable and secure state without requiring full system restarts, which might be restricted by safety or regulatory requirements.

We implement secure software updates and protect them with a TPM. By encrypting the software update with the public SUK we achieve authenticity and confidentiality of security requirement SeR_9 . With the TPM EA policy *TPM2_PolicyCpHash* we also ensure integrity of software updates, enforced by the TPM. This fulfills the integrity part of security requirement SeR_9 . We use further TPM EA policies to restrict access to the SUK, and thus fulfill security requirement SeR_4 (key usage authorization).

During a secure update process, the symmetric key s as well as the firmware update binary are available unencrypted in main

memory for a short period of time. This could pose a potential time window for an attacker to extract the key. The attacker must be able to launch a physical memory extraction attack for that purpose.

We use the Safety Integrity Level 4 (SIL4) safety-certified MILS OS PikeOS to support spatially and temporally separation of compartments. This fulfills safety requirements SaR_1 and SaR_5 . Safety compartments are redundantly executed on two different CPUs in our system architecture, fulfilling safety requirement SaR_2 .

By leveraging a RTOS, we make sure that timing constraints for safety compartments are achieved (safety requirement SaR_4). We provide communication redundancy via distinct communication channels, among other safety-related features. This fulfills safety requirement SaR_3 .

8 FORMAL EVALUATION

In this section, we introduce our results regarding the integrity verification of the Target System as part of the SECURA architecture described in Section 5. Our verification is based on the Security Modeling Framework (SeMF) (cf. [27, 28]). In a first step, we used SeMF to specify a generic model of a Trusted Computing based system. In terms of this system, we formally specified some a priori trust assumptions, e. g., regarding the behavior of the TPM and the RTM complying to the TCG specification [26] that are integrated as hardware-based trust anchors into the system’s OS. We then identified further assumptions that the generic model must comply with to prove fulfillment of certain security properties. We focused on the security properties that can be achieved by a TPM based remote attestation.

Under these assumptions, we were able to show that the components of the TPM protected system can be considered trustworthy at specific moments in the system’s boot cycle. In a second step, we instantiated our generic model by the main components of the SECURA architecture. Hence, our proofs are valid for this system as well, provided it complies with all security assumptions that the proofs are based on.

While a detailed description of the generic model and its verification will be subject of a forthcoming paper, in this paper we focus on its instantiation via the SECURA architecture. In Section 8.1, we present informally the essentials of the SECURA formal model—components, actions, and the most important concepts—that are necessary to understand the verification results. Section 8.4 then introduces the derived security assumptions that the SECURA architecture must satisfy in order for the proofs to be valid. Finally, Section 8.5 outlines how these are used to prove the integrity of the target system.

8.1 Security Modeling Framework SeMF

The Security Modeling Framework (SeMF) makes use of formal languages (see, e. g., [18]). In SeMF, the specification of any kind of cooperating system is based on a prefix-closed formal language whose alphabet is composed of the actions that can happen in the system. More specifically, we use a set of agents (where the term “agent” denotes any entity acting in the system, such as a human being, a device, a part of a device like a component of a system’s hardware platform, etc.), and a set of actions performed by the agents (the letters of the alphabet). The system’s behavior can then

be formally described by a prefix-closed formal language, i. e. by the set of its possible sequences of actions. A system model further comprises the agents' local views (describing which parts of the system behavior agents can actually see) and their initial knowledge which is used to specify system constraints and assumptions.

Security properties are defined in terms of the system specification. The underlying formal semantics then allow formal validation, i. e., allows proving that a specific formal model satisfies specific security properties.

8.2 Components

The components (i. e., agents) of our formal SECURA model correspond to the two main components of the diagram introduced in Fig. 1: the backend that is responsible for remote attestation, and the components of the target system (the system to be attested). Our model comprises the following components: TPM, RTM, UEFI, BootL (denoting the Boot Loader), SepKernel (denoting the RTOS Kernel), AttestS (denoting the Attestation Service), TPMDriver, SecUpdate (denoting the Secure Update), and finally Ver which denotes the part of the backend responsible for appraising a remote attestation.

The trustworthiness of RTM and TPM is implicitly given as they are realized in hardware and act as a trust anchor, while the trustworthiness of all other components of the target system needs to be attested. Further, we assume the Ver as part of the backend to be trustworthy.

We use the concepts explained in Section 5 in the following way:

- While an actual TPM uses a number of several different PCRs in the boot process, we abstract from this and use only PCR0, assuming that extending all measurements to one PCR and attesting this is equivalent to extending to more than one PCR and attesting all of them.
- We further use only one Event Log, denoted by EvLog, instead of differentiating between Boot Log, IMA Log, and Compartment Log. Further, while an Event Log contains not only the hashed executable code of components but other information regarding the code, configuration files, and other elements that influence the system behavior, we abstract from this and consider all elements to be executable code. Thus, each element of EvLog corresponds uniquely to a component of the target system.
- PCRRef denotes a set of trustworthy execution sequences, i. e., a set of reference values with additional information regarding acceptable sequences of started (software) components. The components occurring in such sequences are in principle considered trustworthy. They are trustworthy at a specific moment if started in the order given by the sequence.
- The boot counter managed by the TPM is denoted by *cnt*.
- *fv* denotes a freshness value provided by the verifier for challenge-response based attestation. While TPM 2.0 permits attestation without any freshness value, it is obvious that this does not allow to prove freshness of the attestation evidence. *fv* enables the verifier to identify the boot cycle in which the TPM performed a TPM quote action. Other ways to achieve

freshness include the use of time stamps which, however, require synchronization of clocks or a trustworthy time stamp source (a TPM 2.0 quote includes TPM internal clock and time information).

8.3 Actions

As explained above, in SeMF, system functionality is described via actions performed by components of the system. The actions contain parameters that can be used to specify constraints. These concern the components that can perform a specific action (e. g., the system can only be started by RTM), the order in which specific actions may be processed, etc. However, the formal definition of the actions themselves does not include restrictions. Rather, by formulating them by way of assumptions that honest components must comply with (see Section 8.4), we make such restrictions explicit. This in turn facilitates verification that a specific system complies to all restrictions. In the following, we explain the actions used in the SECURA system in a way that characterizes a desired order of occurrence.

- The target system together with the TPM integrated into the hardware platform is started with the action *init* and shut down with *syshalt*.
- If RTM has started the system, in the next step, using action *extLog* it extends EvLog with the code (i. e., hash digest) of the next component to start. We relax the strict order of the measured boot process explained in Section 5.2.1, and allow RTM to extend and start any target system component except TPM. All target system components except TPM can perform *extLog*.
- With the action *requExt*, all target system components except TPM can request the TPM to extend PCR0 with some code. While some of the components (e. g., RTM, UEFI, TPMDriver) have direct access to the TPM interface, some others (in particular AttestS and SecUpdate) need to use TPMDriver to transfer their request to the TPM.
- With the action *extPCR*, TPM extends PCR0 by some code.
- With the action *acknExt*, TPM acknowledges to have extended PCR0, and with *acknExtRecv*, a target system component (except TPM) receives an acknowledgement, apparently sent by TPM. Note that the actions' parameters only indicate whether the *extend* operation has been successful.
- With the action *start*, any target system component except TPM can start the code of any other target system component (except the codes of TPM and RTM). The component is terminated with *term*. Again, we relax the strict order given by Measured Boot and allow any sequence of starts.
- With *quoteReq*, a component other than the TPM (e. g., the verifier) requests the TPM to perform a quote action (TPM2_Quote command) for PCR0. The request contains a parameter *fv* that serves as a freshness value.
- With the action *quoteSign*, the TPM signs the content of PCR0 concatenated with the current value of the boot counter *cnt*, and a freshness value *fv*. Reception of the *quoteSign* result is modelled by *quoteRecv*. Both actions contain a parameter indicating the component that apparently has initiated *quoteSign*, and that shall receive the action's result.

- With action `readLog`, a target system component other than the TPM (e. g., the verifier) reads the content of the `EvLog`.
- With `quoteVer`, a component other than the TPM (e. g., the verifier) verifies/appraises the data received by `quoteRecv` successfully, e. g., with a positive result.

8.4 Security Assumptions

For proving the security properties of the generic model, we use a set of assumptions. Some of these concern properties related to the behavior of system components that can be taken for granted, others need confirmation that they actually hold in the SECURA system. This concerns, e. g., properties that are not a priori trust assumptions and properties that describe the behavior of the verifier, e. g., its handling of the freshness value fv .

In the following, we introduce assumptions that have been formally derived, and categorize them according to the system layers they are related to. We omit those that are obviously satisfied (but nevertheless need to be specified in the generic model) and focus on those that need explicit verification to hold. Since our work focuses on specific security properties that can be achieved by Trusted Computing technology, we assume all assumptions regarding the underlying cryptographic mechanisms to be satisfied. Finally, we outline how the assumptions are used to prove the integrity of the target system.

8.4.1 Assumptions Concerning the SECURA Platform.

Asm₁ In every state of the system, RTM is trustworthy, and only RTM can start the system, i. e., perform the action `init`. With system start, `EvLog` is reset to its initial value (i. e. it is empty). Directly after having initiated the system, the next actions are as follows (unless the system is halted): RTM extends `EvLog` with some code and requests the TPM to extend the PCR with this code. Then, TPM performs the requested extension.

8.4.2 Assumptions Concerning the SECURA Software. If a component has direct access to the TPM interface—as is the case for, e. g., RTM, UEFI, and `TPMDriver`—, by its very nature, there is a unique relation between this component’s `requExt` action, the `extPCR`, and `acknExt` actions by the TPM as well as the final `acknExtRecv` action by the requester. However, this is not necessarily the case for components without direct access to the TPM interface (e. g., `AttestS` and `SecUpdate`). Hence, for these components the SECURA system must guarantee that the initiator of an `extPCR` action receives the TPM’s respective acknowledgement. If this assumption is violated, the acknowledgement by the TPM (which contains no data that could be linked to the initiation action) may be transferred to a different component that may have initiated a different `extPCR` action. This in turn could cause the recipient of the acknowledgement to perform a `start` action for potentially malicious code that had not been extended to the PCR, and which would consequently not be recognizable by a later attestation.

This rule is integrated into the “first measure then start” principle explained in Section 5.2, and formalized in the following assumption. While it may seem obsolete—since none of the compartment components except `TPMDriver` is supposed to request any `extPCR` action—, it is actually critical since a manipulated/compromised

component may very well not comply to this. Further, an active attacker on the bus between a compute unit (e. g., the CPU) and the TPM may be able to tamper with the data sent to and from the TPM. However, this can be mitigated by using end-to-end encrypted sessions between the component that performs the `extPCR` action and the TPM.

Asm₂ An honest component only starts the code of another component (action `start`) if it has received an acknowledgement (`acknExtRecv`) authentically originating from the TPM to have extended `PCR0`. Before acknowledging PCR extension (`acknExt`), the TPM must have actually extended `PCR0` with some code (`extPCR`), and this action must have been initiated by the honest component’s request for PCR extension by this code (`requExt`). Most importantly, all actions must happen in only one of the honest component’s execution contexts (i. e., they may not occur in different execution contexts).

In reality, in the static part of the boot process, a trustworthy component extends a PCR with the code of the next component to start, then boots it, and passes control to it. If, for whatever reason, the component terminates before having booted the next one, there is no further PCR extension, and the whole boot process comes to a stop. Further, in the dynamic phase, many systems (e. g., SECURA) use a central bootloader that performs all further boot actions. Again, if this bootloader terminates, no further components are extended and booted. In consequence, if only trustworthy components are involved, the sequence of boot actions is equivalent to the sequence of extend PCR actions (`extPCR`):

Asm₃ We assume that, before a new request to PCR extension (`extPCR`) occurs, the code of the previous one has been started.

8.4.3 Assumptions Concerning the SECURA Backend.

Asm₄ fv represents a freshness value generated by the verifier `Ver`. Our model uses a random number, i. e., data that is assumed to be generated only once and that is practically infeasible to be guessed. As a consequence, a request for attestation by a legitimate agent is the first usage of a specific fv value (afterward, a malicious agent can use it any number of times). Consequently, any other agent can use fv only after it has been used by `Ver` in an action `quoteReq`. Further, the verifier `Ver` uses each freshness value fv only once in a request for attestation (`quoteReq`), and once in a quote verification action (`quoteVer`).

Asm₅ An action `quoteVer` by the verifier `Ver` represents a positive verification (i. e., acceptance) of a `quoteSign` result. When processing this action, the verifier must ensure that `PCR0` corresponds to the content of `EvLog`, and that this in turn corresponds to a trustworthy execution sequence.

Asm₆ For challenge-response based attestation (which is what we based our formal analysis on), the verifier only receives (i. e., accepts) the result of a `quoteSign` action if it has requested the TPM to perform one.

8.4.4 Assumptions Concerning the TPM.

Asm₇ With system start, the TPM resets `PCR0` to its initial value zero. After that, the value can only be changed through an

action `extPCR`. Further, at the very first start of the TPM, the reboot counter `cnt` is set to zero (and also the restart counter). Then, with each new boot cycle, the TPM increases it by one and keeps this value during the complete boot cycle. Within one boot cycle, `cnt` does not change. No other component can change the value of `cnt`.

Asm₈ TPM only performs an action `quoteSign` if requested to do so. The `quoteReq` action directly triggering the `quoteSign` action (i. e., the request at the TPM interface) occurs in the same boot cycle as the `quoteSign` action. The `PCR0` content represents all PCR extensions of this particular boot cycle until the point the `quoteSign` action has been performed. In other words, there is a one-to-one relation between the codes that make up the content of the PCR (identified by `EvLog`) and the actions `extPCR` that have been performed by the TPM until the `quoteSign` action.

Asm₉ The basic security property provided by the TPM extends the trust of the verifier regarding the first i components of `EvLog` to the next one in the row. More precisely, it can be described as follows:

Assume Ver performs a `quoteVer` action verifying the result of a specific `quoteSign` action performed by TPM with a positive result, and assume the following properties to hold:

- (1) The first $i + 1$ components of `EvLog` constitute a trustworthy execution sequence.
- (2) The first i components of `EvLog` are already proven to be trustworthy in the boot cycle of the specific `quoteSign` action. More precisely, in the system phase starting with initiating this boot cycle and ending with the `quoteSign` action, these components act honestly.
- (3) Any action in which TPM extends `PCR0` with `codei+1` must have been triggered by a trustworthy component. Further, `codei` must have been started between the two `PCR0` extensions with `codei` and `codei+1`.

Then, `codei+1` can be trusted by Ver in this particular boot cycle to act honestly when acting before the `quoteSign` action. In other words, the trust of Ver is extended from the components corresponding to `code1, ..., codei` to the one corresponding to `codei+1`.

8.5 Proof Sketch

Assuming the above listed assumptions to hold for the generic system model, we have proven the integrity of the target system components until the occurrence of a `quoteSign`, which therefore also holds for any instantiation of the generic model and thus in particular for the SECURA architecture. In the following, we outline the main ideas of the proof.

In a first step, it can be shown that, when a `quoteVer` action is performed by the verifier, the corresponding `quoteSign` action always happens in the system phase that starts with the respective `quoteReq` action by the verifier, and ends with the verifier's `quoteVer` action. The proof relies on the properties of the freshness value `fv` included in `quoteReq` which never can have been used before.

Our proof highlights that without additional assumptions it cannot be shown that all three actions `quoteReq`, `quoteSign`, and `quoteVer` occur in the same boot cycle. The system may very well be halted, newly started and the `quoteSign` action triggered by a `quoteReq` action of a malicious component, repeating Ver's freshness value `fv`. Further, to prove that `quoteSign` and `quoteVer` occur in the same boot cycle, Ver would need to know the value of the boot cycle counter `cnt` at the moment of its `quoteVer` action. However, the only information Ver has is the value of `cnt` in the boot cycle of the `quoteSign` action and its value in past `quoteSign` actions.

It may very well be that after `quoteSign` and before the verifier's corresponding `quoteVer` action the system is halted and started again. The system may even be in the same state regarding the PCR value at the moment of Ver's action `quoteVer`, yet the counter would have increased without Ver noticing. It depends on the particular system to which extent a verifier can identify a time gap between its actions `quoteReq` and `quoteVer` that would indicate that a malicious component has blocked its `quoteReq` action and/or delayed submission of the `quoteSign` result. A system that needs to prohibit such gaps—and thus requires occurrence of `quoteReq`, `quoteSign`, and `quoteVer` in the same boot cycle—needs to take further measures by for example establishing an authorization session between the TPM and the verifier.

For specific actions, e. g., software update, assurance is required that the system is in a trustworthy state at the point the action is performed. This can be achieved as follows: The verifier initiates and verifies/appraises a remote attestation, then the action in question is performed, and then the verifier again initiates and performs a remote attestation. If both `quoteVer` actions attest the same PCR content (or in case the updated software has been started already, the PCR would have changed) and `cnt` value, the verifier can be sure that the system has not been changed between the two attestation processes and in particular has not been halted and started again (i. e., rebooted). We have not formally addressed this issue further.

In a second proof step, it can be shown that after the occurrence of an action `quoteVer`, all components related to the codes contained in the PCR of the corresponding action `quoteSign` can be trusted by the verifier to act honestly until `quoteSign` occurs.

The essentials of the proof are that:

- (1) the target system is started by RTM, which is considered trustworthy and thus always extends a code to the PCR before booting it, and that
- (2) all PCR entries of the `quoteSign` action correspond uniquely to an `extPCR` action by TPM.

By *Asm₉*, the trust into RTM can be transferred from RTM to the first component of PCR, and then step by step from one component to the next one, thus covering all components of PCR.

These two statements finally allow proving the main result: After having performed a `quoteVer` action, the verifier trusts any component to be honest that acts within the boot cycle of the corresponding `quoteSign` action, and before this action occurs.

The proof of this statement considers an arbitrary action a . In case a is performed by RTM, it immediately follows the assertion. In case it is performed by some other component comp, it must have happened in the component's execution context. This can be

started by RTM which by the property shown in the second proof step allows transferring the trust into RTM to trust into comp. If on the other hand comp is started by another component $comp'$, we now consider this component's execution context. Again, it can be started by RTM which transfers the trust to $comp'$, and which in turn, being honest, transfers the trust to comp, or it is started by yet another component. Since all sequences in the boot cycle until quoteSign are finite, we only have finitely many start actions by components different from RTM. So, this line of argument implies that the trust into RTM can be transferred to each of the components involved in starting comp, thus proving the trustworthiness of comp.

9 CONCLUSION AND FUTURE WORK

In this paper, we introduced the SECURA architecture to enable the operation of non-safety-certified security measures on safety-certified OT systems. The focus of our work was integrity protection (and verification) of a target system using a TPM in a combination with a RTOS. We used the TPM as basis for a measured boot, a double heartbeat remote attestation protocol, a TEE based local attestation and a cyber-resiliency watchdog, a secure update protocol, and vulnerability monitoring. Our concept also involves the required backend and the communication between the target system and backend. We evaluated SECURA based on identified security and safety requirements. Additionally, we formally evaluated the remote attestation process as a central aspect of our concept.

Future work includes the application of the SECURA architecture to other domains, such as smart grid, automotive IoT, and avionics. With the evolution of post-quantum cryptography and its availability in TPMs, more sophisticated secure communication protocols and key exchange mechanisms may be developed. Further, we aim to enhance the SECURA architecture by integrating quantum-safe watchdog timer protocols, as researched in [16], to bolster the resilience and security of our system against emerging quantum computing threats. Formal verification of the secure update approach as well as its application to safety compartments promise further interesting directions of research.

ACKNOWLEDGMENTS

This research work was supported by the National Research Center for Applied Cybersecurity ATHENE [34]. ATHENE is funded jointly by the German Federal Ministry of Education and Research and the Hessian Ministry of Higher Education, Research and the Arts. Further, this research work was supported by the projects TRUSTnet [11] (ID 16KIS1787), VE-ASCOT [40] (ID 16ME0274), and HASELNUSS [20] (ID 16KIS0597K) from the German Federal Ministry of Education and Research as well as the projects RESISTANT [21] (ID 20D2116B) and SecDER [10] (ID 03EI4028A) from the German Federal Ministry for Economic Affairs and Climate Action.

REFERENCES

- [1] Irune Agirre, Mikel Azkarate-Askasua, Asier Larrucea, Jon Perez, Tullio Vardanega, and Francisco Cazorla. 2015. A Safety Concept for a Railway Mixed-Criticality Embedded System Based on Multicore Partitioning. In *IEEE International Conference on Computer and Information Technology; Ubiquitous Computing and Communications; Dependable, Autonomic and Secure Computing; Pervasive Intelligence and Computing*. <https://doi.org/10.1109/cit/iuicc/dasc/picom.2015.268>
- [2] Arm Limited. 2024. *Cortex-R82 will power next-generation storage devices - Internet of Things (IoT) blog - Arm Community blogs - Arm Community*. Arm Limited. Retrieved June 16, 2024 from <https://community.arm.com/arm-community-blogs/b/internet-of-things-blog/posts/arm-cortex-r82-high-performance-64bit-realtime-applications-processing>
- [3] Chet Babla. 2021. *Arm Cortex-R52+ Enhances Time-critical Automotive Designs - Arm Newsroom*. VP Automotive, Arm Limited. Retrieved June 16, 2024 from <https://newsroom.arm.com/blog/automotive-cortex-r>
- [4] Diethelm Bienhaus, Lukas Jäger, Roland Rieke, and Christoph Krauß. 2019. Gateway for Industrial Cyber-Physical Systems with Hardware-Based Trust Anchors. In *Intelligent Distributed Computing XIII, 13th International Symposium on Intelligent Distributed Computing, IDC 2019, St. Petersburg, Russia, 7-9 October, 2019*. 521–528. https://doi.org/10.1007/978-3-030-32258-8_61
- [5] Ainara Bilbao, Irune Yarza, Jose Montero, Mikel Azkarate-askasua, and Nera Gonzalez. 2017. A railway safety and security concept for low-power mixed-criticality systems. In *IEEE 15th International Conference on Industrial Informatics (INDIN)*. 59–64. <https://doi.org/10.1109/INDIN.2017.8104747>
- [6] Henk Birkholz, Michael Eckel, Wei Pan, and Eric Voit. 2024. *Reference Interaction Models for Remote Attestation Procedures*. Internet-Draft draft-ietf-rats-reference-interaction-models-09. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-ietf-rats-reference-interaction-models/09/> Work in Progress.
- [7] Henk Birkholz, Christoph Krauß, Maria Zhdanova, Don Kuzhiyelil, Tolga Arul, Markus Heinrich, Stefan Katzenbeisser, Neeraj Suri, Tsvetoslava Vateva-Gurova, and Christian Schlehuber. 2018. A Reference Architecture for Integrating Safety and Security Applications on Railway Command and Control Systems. In *4th International Workshop on MILS: Architecture and Assurance for Secure Systems, (MILS@DSN 2018)*. <https://doi.org/10.5281/zenodo.1314095>
- [8] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. Remote ATtestation procedureS (RATS) Architecture. RFC 9334. <https://doi.org/10.17487/RFC9334>
- [9] G. Coker, J. Guttman, P. Loscocco, A. Herzog, J. Millen, B. O'Hanlon, J. Ramsdell, A. Segall, J. Sheehy, and B. Sniffen. 2011. Principles of Remote Attestation. *Int. J. Inf. Secur.* 10, 2 (June 2011), 63–81.
- [10] DECOIT GmbH & Co. KG. 2022. *SecDER Project – AI-based Detection and Resilient Prevention of Cyber-Attacks and Technical Disruptions in Virtual Power Plants and Decentralized Energy Systems*. German Federal Ministry for Economic Affairs and Climate Action. <https://secder-project.de/>
- [11] DECOIT GmbH & Co. KG. 2023. *TRUSTnet Project – Trusted Core Network in Real and Simulated Industry 4.0 Environments*. German Federal Ministry of Education and Research. <https://trustnet-project.de/>
- [12] DKE. 2015. DKE: Elektrische Bahn-Signalanlagen – Teil 104: Leitfaden für die IT-Sicherheit auf Grundlage der IEC 62443.
- [13] D. Dolev and A. C. Yao. 1981. On the Security of Public Key Protocols. In *Proceedings of the 22Nd Annual Symposium on Foundations of Computer Science (SFCS '81)*. IEEE Computer Society, Washington, DC, USA, 350–357. <https://doi.org/10.1109/SFCS.1981.32>
- [14] Michael Eckel, Florian Fenzl, and Lukas Jäger. 2024. Towards Practical Hardware Fingerprinting for Remote Attestation. In *39th International Conference on ICT Systems Security and Privacy Protection (Edinburgh, UK) (IFIP SEC 2024)*, Steven Furnell, Sokratis K. Katsikas, Konstantinos Markantonakis, and Nikolaos Pitropakis (Eds.). Springer International Publishing, Cham, 14 pages. https://doi.org/10.1007/978-3-030-58201-2_14
- [15] Michael Eckel, Andreas Fuchs, Jürgen Repp, and Markus Springer. 2020. Secure Attestation of Virtualized Environments. In *ICT Systems Security and Privacy Protection (Maribor, Slovenia) (IFIP SEC 2020)*, Marko Hölbl, Kai Rannenberg, and Tatjana Welzer (Eds.). Springer International Publishing, Cham, 203–216. https://doi.org/10.1007/978-3-030-58201-2_14
- [16] Michael Eckel, Tanja Gutsche, Hagen Lauer, and André Rein. 2023. A Generic IoT Quantum-Safe Watchdog Timer Protocol. In *Proceedings of the 18th International Conference on Availability, Reliability and Security (Benevento, Italy) (ARES 2023)*. Association for Computing Machinery (ACM), New York, NY, USA, Article 90, 10 pages. <https://doi.org/10.1145/3600160.3605169>
- [17] Michael Eckel, Don Kuzhiyelil, Christoph Krauß, Maria Zhdanova, Stefan Katzenbeisser, Jasmin Čosić, Matthias Drod, and Jean-Jacques Pitrolle. 2021. Implementing a Security Architecture for Safety-Critical Railway Infrastructure. In *2021 International Symposium on Secure and Private Execution Environment Design (SEED) (SEED 2021)*. 215–226. <https://doi.org/10.1109/SEED51797.2021.00033>
- [18] S. Eilenberg. 1974. *Automata, Languages and Machines*. Academic Press, New York.
- [19] Fraunhofer Institute for Secure Information Technology SIT. 2019. *CHARRA: CHALLENGE-Response based Remote Attestation with TPM 2.0*. <https://github.com/Fraunhofer-SIT/charra>
- [20] Fraunhofer Institute for Secure Information Technology SIT. 2021. *HASELNUSS Project – Hardware-based Security Platform for Railway Command and Control Systems*. German Federal Ministry of Education and Research. <https://haselnuss-projekt.de/>
- [21] Fraunhofer Institute for Secure Information Technology SIT. 2023. *RESISTANT Project – Resilient Zero-Trust Avionics Platform supported by Digital Security Twins*

- and Aircraft-SOCs. German Federal Ministry for Economic Affairs and Climate Action. <https://www.sit.fraunhofer.de/de/resistant/>
- [22] Andreas Fuchs, Henk Birkholz, Ira McDonald, and Carsten Bormann. 2022. *Time-Based Uni-Directional Attestation*. Internet-Draft draft-birkholz-rats-tuda-07. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-birkholz-rats-tuda/> Work in Progress.
- [23] Andreas Fuchs, Dustin Kern, Christoph Krauß, and Maria Zhdanova. 2020. TrustEV: Trustworthy Electric Vehicle Charging and Billing. In *SAC 2020, SEC - Computer Security*.
- [24] Andreas Fuchs, Christoph Krauß, and Jürgen Repp. 2016. Advanced Remote Firmware Upgrades Using TPM 2.0. In *ICT Systems Security and Privacy Protection*, Jaap-Henk Hoepman and Stefan Katzenbeisser (Eds.). Springer International Publishing, Cham, 276–289.
- [25] Georgios Gkoktsis, Hagen Lauer, and Lukas Jäger. 2023. Risk Assessments in Virtual Power Plants with NESCOR Criteria, Practical Application, Advantages and Disadvantages. In *Proceedings of the 18th International Conference on Availability, Reliability and Security* (Benevento, Italy) (ARES '23). Association for Computing Machinery, New York, NY, USA, Article 50, 11 pages. <https://doi.org/10.1145/3600160.3605179>
- [26] Trusted Computing Group. 09/29/2016. TPM 2.0 Library Specification. <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [27] S. Gürgens, P. Ochsenschläger, and C. Rudolph. 2002. Authenticity and provability - a formal framework. In *Infrastructure Security Conference InfraSec 2002 (Lecture Notes in Computer Science, Vol. 2437)*. Springer Verlag, 227–245.
- [28] S. Gürgens, P. Ochsenschläger, and C. Rudolph. 2005. On a formal framework for security properties. *International Computer Standards & Interface Journal (CSI), Special issue on formal methods, techniques and tools for secure and reliable applications* 27, 5 (June 2005), 457–466.
- [29] Markus Heinrich, Tsvetoslava Vateva-Gurova, Tolga Arul, Stefan Katzenbeisser, Neeraj Suri, Henk Birkholz, Andreas Fuchs, Christoph Krauß, Maria Zhdanova, Don Kuzhiyelil, Sergey Tverdyshev, and Christian Schlehuber. 2019. Security Requirements Engineering in Safety-Critical Railway Signalling Networks. *Security and Communication Networks* vol. 2019, Article ID 8348925 (2019), 14 pages. <https://doi.org/10.1155/2019/8348925>
- [30] Markus Heinrich, Jannik Vieten, Tolga Arul, and Stefan Katzenbeisser. 2018. Security Analysis of the RaSTA Safety Protocol. In *IEEE International Conference on Intelligence and Security Informatics (ISI)*. 199–204. <https://doi.org/10.1109/ISI.2018.8587371>
- [31] Lukas Jäger and Dominik Lorych. 2021. Remote Attestation Extended to the Analog Domain. In *Proceedings of the 16th International Conference on Availability, Reliability and Security* (Vienna, Austria) (ARES '21). Association for Computing Machinery, New York, NY, USA, Article 4, 10 pages. <https://doi.org/10.1145/3465481.3465762>
- [32] Christoph Krauß, Maria Zhdanova, Michael Eckel, Katzenbeisser Stefan, Markus Heinrich, Don Kuzhiyelil, Jasmin Čosić, and Matthias Drodts. 2021. A Security Architecture for Protecting Safety-Critical Railway Infrastructure. In *embedded world Conference 2021 DIGITAL Proceedings* (Nuremberg, Germany) (ewC 2021). WEKA FACHMED- DIEN GmbH, Richard-Reitzner-Allee 2, 85540 Haar, Germany, 296–299. <https://shop.weka-business-communication.com/Sonderpublikationen-dsb/embedded-world-Conference-2021-DIGITAL-Proceedings.html>
- [33] Igor Lopez and Marina Aguado. 2015. Cyber security analysis of the european train control system. *IEEE Communications Magazine* 53, 10 (2015), 110–116.
- [34] National Research Center for Applied Cybersecurity ATHENE c/o Fraunhofer Institute for Secure Information Technology SIT. 2024. *National Research Center for Applied Cybersecurity ATHENE*. National Research Center for Applied Cybersecurity ATHENE c/o Fraunhofer Institute for Secure Information Technology SIT. <https://athene-center.de/>
- [35] Marco Rocchetto and Nils Ole Tippenhauer. 2016. CPDY: Extending the Dolev-Yao Attacker with Physical-Layer Interactions. In *Formal Methods and Software Engineering*, Kazuhiro Ogata, Mark Lawford, and Shaoying Liu (Eds.). Springer International Publishing, Tokyo, Japan, 175–192. https://doi.org/10.1007/978-3-319-47846-3_12
- [36] Christian Schlehuber, Markus Heinrich, Tsvetoslava Vateva-Gurova, Stefan Katzenbeisser, and Neeraj Suri. 2017. Challenges and Approaches in Securing Safety-Relevant Railway Signalling. In *IEEE European Symposium on Security and Privacy Workshops*. 139–145. <https://doi.org/10.1109/EuroSPW.2017.63>
- [37] Christian Spinnler, Torsten Labs, and Norman Franchi. 2024. SoK: A Taxonomy for Hardware-Based Fingerprinting in the Internet of Things. In *Proceedings of the 19th International Conference on Availability, Reliability and Security* (Vienna, Austria) (ARES '24). Association for Computing Machinery, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3664476.3670872>
- [38] Trusted Computing Group 2024. *Trusted Platform Module Library - Part 1: Architecture* (family 2.0, level 00, revision 01.83 ed.). Trusted Computing Group.
- [39] Trusted Computing Group 2024. *Trusted Platform Module Library - Part 3: Commands* (family 2.0, level 00, revision 01.83 ed.). Trusted Computing Group.
- [40] WIBU-SYSTEMS AG. 2021. *VE-ASCOT – Advanced Security for Chains of Trust*. German Federal Ministry of Education and Research. <http://ascot-trust.info/>
- [41] Menghua Xu and Huibing Zhao. 2012. Simulation and verification of rssp-ii safety communication protocol. In *2012 Second International Conference on Business Computing and Global Informatization*. IEEE, 731–734.